

OPERATING SYSTEMS

Complete Study Notes

Chapters 4, 6 & 7

Threads & Concurrency | Synchronization Tools | Classical Synchronization Problems

Silberschatz, Galvin & Gagne, 10th Edition

Chapter 4: Threads & Concurrency

(Silberschatz, Galvin & Gagne, 10th Edition)

1. Motivation

- Most modern applications are multithreaded; threads run within an application.
- Multiple tasks within one app can be separate threads:
- Update display
- Fetch data
- Spell checking
- Answer a network request
- Process creation is heavy-weight; thread creation is light-weight.
- Threads can simplify code and increase efficiency.
- Kernels themselves are generally multithreaded.

2. Threads vs Processes

- A single-threaded process: one program counter, one stack, one set of registers.
- A multithreaded process: threads share code section, data section, heap, open files, signals - but each thread has its own registers, program counter, and stack.

Multithreaded Server Architecture

Client sends a request → server creates (or assigns) a thread to service it → server resumes listening for more requests.

3. Benefits of Multithreading

1. Responsiveness - execution continues even if part of the process is blocked; crucial for user interfaces.
 - Real life: You can still scroll a webpage while images load. Without threads, the UI would freeze - the dreaded "Not Responding."
2. Resource sharing - threads share the process's resources automatically; easier than shared memory or message passing between processes.
3. Economy - cheaper than process creation; thread switching has lower overhead than full context switching.
 - Real life: Hiring a roommate vs. building a new house.
4. Scalability - a process can exploit multicore architectures: threads run in parallel on different cores.

4. Multicore Programming

Multicore/multiprocessor systems put pressure on programmers. Challenges:

- Dividing activities - finding tasks that can run in parallel
- Balance - tasks should do roughly equal work
- Data splitting - dividing data across tasks
- Data dependency - when one task needs another's result, execution must be synchronized
- Testing and debugging - many possible interleavings make bugs hard to reproduce

Concurrency vs Parallelism

- Parallelism - the system can perform more than one task simultaneously (needs multiple cores).
- Concurrency - more than one task making progress (possible on a single core via the scheduler interleaving them).
- Single core: T1, T2, T3, T4 interleaved in time slices. Multicore: T1 and T3 literally at the same instant on different cores.

Types of Parallelism

- Data parallelism - distribute subsets of the same data across cores, same operation on each.
- Real life: Applying a photo filter - each core processes a quarter of the pixels.
- Task parallelism - distribute threads (different operations) across cores.
- Real life: One core computes the average of a spreadsheet column while another computes the max of the same data. Or in a kitchen: data parallelism = four cooks each chopping a quarter of the onions; task parallelism = one chops, one stirs, one plates.

5. User Threads and Kernel Threads

- User threads - managed by a user-level threads library. Three primary libraries: POSIX Pthreads, Windows threads, Java threads.
 - Kernel threads - supported by the kernel. Virtually all general-purpose OSes: Windows, Linux, macOS, iOS, Android.
- The relationship between user and kernel threads is defined by the multithreading model.

6. Multithreading Models

Many-to-One

- Many user-level threads → one kernel thread.
- One thread blocking causes all to block.
- Threads can't run in parallel on multicore (only one may be in the kernel at a time).
- Few systems still use it. Examples: Solaris Green Threads, GNU Portable Threads.

One-to-One

- Each user thread → its own kernel thread.
- More concurrency; creating a user thread creates a kernel thread.
- Number of threads per process sometimes restricted due to overhead.
- Examples: Windows, Linux.

Many-to-Many

- Many user threads mapped to many ($\leq$) kernel threads.
- OS creates a sufficient number of kernel threads.
- Windows with the ThreadFiber package; otherwise not very common.

Two-level Model

- Like many-to-many, but also allows a user thread to be bound to a specific kernel thread.

7. Thread Libraries

- A thread library provides the programmer with an API for creating and managing threads.
- Two ways to implement:
 1. Library entirely in user space (no kernel involvement)
 2. Kernel-level library supported by the OS

8. Java Threads

- Managed by the JVM; typically implemented using the underlying OS's thread model.
- Creating threads:
 - Extend the Thread class, or
 - Implement the Runnable interface ← standard practice:

```
class Task implements Runnable {
    public void run() { /* work here */ }
}
Thread worker = new Thread(new Task());
worker.start();           // create & run
worker.join();            // wait for it to finish
```

Java Executor Framework

- Rather than explicitly creating threads, use the Executor interface:


```
ExecutorService pool = Executors.newSingleThreadExecutor();
pool.execute(new Task());           // run a task
Future<Integer> result = pool.submit(new Callable<Integer>() { ... });
result.get();                       // retrieve the computed value
```
- Separates task creation from task execution; supports returning results via Callable/Future.

Key Terms Checklist

Thread, single- vs multithreaded process, shared (code/data/heap/files) vs per-thread (stack/registers/PC), responsiveness, resource sharing, economy, scalability, concurrency vs parallelism, data vs task parallelism, dividing activities/balance/data splitting/data dependency/debugging, user threads, kernel threads, Pthreads/Windows/Java libraries, many-to-one, one-to-one, many-to-many, two-level

model, Runnable, join(), Executor framework, Callable/Future, thread pool.

Chapter 6: Synchronization Tools

(Silberschatz, Galvin & Gagne, 10th Edition)

1. Background

- Processes execute concurrently and may be interrupted at any time, partially completing execution.
- Concurrent access to shared data can cause data inconsistency.
- Maintaining consistency requires mechanisms ensuring orderly execution of cooperating processes.
- Recall the bounded-buffer counter race from Chapter 3 - producer and consumer both updating counter concurrently.

Race condition in the kernel too

Processes P0 and P1 both call `fork()`; both read the kernel variable `next_available_pid` before either updates it → two different processes get the same pid. Without a mechanism preventing simultaneous access, kernel data corrupts just like user data.

2. The Critical-Section Problem

- System of n processes $\{p_0, \dots, p_{n-1}\}$; each has a critical section (CS) of code where it changes common variables, updates a table, writes a file, etc.
- When one process is in its CS, no other may be in its CS.
- The problem: design a protocol to ensure this.
- General structure of each process:

```
while (true) {
    [entry section]    ← ask permission to enter
    critical section
    [exit section]     ← announce leaving
    remainder section
}
```

Three requirements for a correct solution

1. Mutual Exclusion - if P_i is in its CS, no other process is in its CS.
2. Progress - if no process is in its CS and some want in, the selection of who enters next cannot be postponed indefinitely (only processes not in their remainder sections vote).
3. Bounded Waiting - a bound on how many times others can enter their CS after a process requests entry and before it's granted. (Assume nonzero speeds; no assumptions about relative speed.)

3. Interrupt-Based Solution

- Entry section: disable interrupts; exit section: enable interrupts.
- Problems:
- What if the CS runs for an hour? (Everything else freezes.)
- Processes could starve.
- What if there are two CPUs? Disabling interrupts on one doesn't stop the other.

4. Software Solution 1 (turn variable)

- Two processes; assume load and store are atomic (can't be interrupted).
- Shared: `int turn` - whose turn to enter; initially `i`.
- Process P_i :

```
while (true) {
    while (turn == j); /* wait */
    /* critical section */
    turn = j;
    /* remainder section */
}
```

- Mutual exclusion: preserved (P_i enters only if `turn == i`, and `turn` can't be both values at once).
- Progress: violated - strict alternation. If it's P_j 's turn but P_j doesn't want in (busy in its remainder), P_i is stuck even though the CS is free.
- Bounded waiting: satisfied, but progress failure sinks it.

5. Peterson's Solution

- Two processes; shared variables:

- int turn - whose turn it is
- boolean flag[2] - flag[i] = true means Pi is ready to enter
- Process Pi:


```

while (true) {
    flag[i] = true;           /* I want in */
    turn = j;                /* but you first, politely */
    while (flag[j] && turn == j)
        ;                    /* wait */
    /* critical section */
    flag[i] = false;
    /* remainder section */
}

```

- Provably meets all three requirements:

1. Mutual exclusion - Pi enters only if flag[j] == false or turn == i
2. Progress ✓
3. Bounded waiting ✓

Peterson's Solution and Modern Architecture — why it breaks

- Modern processors/compilers reorder operations with no dependencies to improve performance.
 - Fine for single-threaded code (same result), but for multithreaded code reordering produces inconsistent/unexpected results.
- Example. Shared: boolean flag = false; int x = 0;
- Thread 1: while (!flag); print x
 - Thread 2: x = 100; flag = true;
 - Expected output: 100. But x = 100 and flag = true are independent → may be reordered → Thread 1 can print 0!
 - In Peterson's solution, reordering the flag[i] = true and turn = j writes can let both processes into their CS simultaneously.

6. Memory Barriers

- A memory model = the memory guarantees an architecture makes to programs:
- Strongly ordered - a modification by one processor is immediately visible to all others
- Weakly ordered - modifications may not be immediately visible
- A memory barrier is an instruction forcing any change in memory to be propagated (made visible) to all other processors.
- When a barrier executes, all loads and stores complete before any subsequent load/store - even if instructions were reordered, stores are visible to other processors before later operations.

Fixed example:

<pre> Thread 1: while (!flag) memory_barrier(); print x </pre>	<pre> Thread 2: x = 100; memory_barrier(); flag = true; </pre>
--	--

Thread 1 is guaranteed to load flag before x; Thread 2 guarantees x = 100 happens-before flag = true → output is 100.

7. Hardware Instructions

Many systems provide hardware support for the CS problem. On uniprocessors, disabling interrupts works but is too inefficient on multiprocessors (not scalable). Instead: atomic (uninterruptible) instructions that test-and-modify a word, or swap two words.

test_and_set

```

boolean test_and_set(boolean *target) {
    boolean rv = *target; /* return original value */
    *target = true;       /* set to true */
    return rv;
} /* executed ATOMICALLY */

```

Usage (shared boolean lock = false):

```

do {
    while (test_and_set(&lock))
        ; /* spin */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);

```

- Provides mutual exclusion, but not bounded waiting (an unlucky process can lose the race forever).

compare_and_swap (CAS)

```

int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)

```

```

        *value = new_value; /* swap only if it matches expected */
    return temp;
} /* executed ATOMICALLY */

```

Usage (shared int lock = 0):

```

while (true) {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* spin */
    /* critical section */
    lock = 0;
    /* remainder section */
}

```

- Mutual exclusion ✓; bounded waiting still ✗ in this simple form.

Bounded-waiting version with CAS

Shared: boolean waiting[n] (all false), int lock = 0.

```

while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = 0; /* nobody waiting – free the lock */
    else
        waiting[j] = false; /* pass the CS directly to Pj */
    /* remainder section */
}

```

- On exit, a process scans forward cyclically for the next waiting process and hands the CS to it → every waiter gets in within n–1 turns → bounded waiting achieved.

8. Atomic Variables

- Instructions like CAS are building blocks for other tools.
- An atomic variable provides atomic updates on basic types (integers, booleans).
- E.g., increment(&sequence;) increments without interruption, implemented with CAS:

```

void increment(atomic_int *v) {
    int temp;
    do { temp = *v; }
    while (temp != compare_and_swap(v, temp, temp+1));
}

```

(Keep retrying until nobody changed the value between the read and the swap.)

9. Mutex Locks

- Hardware solutions are complicated and inaccessible to application programmers → OS designers provide software tools.
- Mutex lock - simplest: a boolean indicating whether the lock is available.
- Protect a CS: acquire() the lock first, release() after.
- acquire()/release() calls must be atomic - usually built with hardware atomic instructions like CAS.
- This solution requires busy waiting → such a lock is called a spinlock.

```

while (true) {
    acquire lock
    critical section
    release lock
    remainder section
}

```

10. Semaphores

- More sophisticated than mutex locks. Semaphore S = integer variable, accessed only via two atomic operations:
- wait(S) (originally P): while (S <= 0); S--;
- signal(S) (originally V): S++;
- Counting semaphore - value ranges over an unrestricted domain (counts available resources).
- Binary semaphore - value 0 or 1; same as a mutex lock. A counting semaphore can be implemented from binary ones.

Usage patterns

1. Mutual exclusion: semaphore mutex = 1:

```
wait(mutex); CS signal(mutex);
```

2. Ordering (S1 before S2): semaphore synch = 0:

```
P1: S1; signal(synch);
P2: wait(synch); S2;
```

Implementation

- Must guarantee no two processes execute wait() and signal() on the same semaphore at the same time → the implementation itself is a CS problem (wait/signal code placed in a critical section).

- Busy waiting inside this implementation is short and rare - but applications may spend lots of time in critical sections, so busy waiting there isn't good.

Implementation with NO busy waiting

Each semaphore has an associated waiting queue; two kernel operations:

- block() - place the invoking process on the waiting queue (state → waiting)

- wakeup(P) - move P from the waiting queue to the ready queue

```
typedef struct {
    int value;
    struct process *list;
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

(Negative value = number of processes waiting.)

Problems with Semaphores — incorrect use

- signal(mutex) ... wait(mutex) - reversed → mutual exclusion broken

- wait(mutex) ... wait(mutex) - double wait → deadlock

- Omitting wait() and/or signal() → violation or deadlock

11. Monitors

- A high-level abstraction: an abstract data type whose internal variables are accessible only by code within its procedures.

- Only one process may be active within the monitor at a time - mutual exclusion is automatic.

```
monitor monitor-name {
    // shared variable declarations
    procedure P1 (...) { ... }
    ...
    initialization code (...) { ... }
}
```

- Basic implementation with semaphores: semaphore mutex = 1; every procedure body wrapped in wait(mutex) ... signal(mutex).

Condition Variables

- condition x, y; with two operations:

- x.wait() - the invoking process is suspended until someone calls x.signal()

- x.signal() - resumes one waiting process; if none is waiting, it has no effect (unlike a semaphore signal, which always increments!)

Ordering example (S1 before S2) - monitor with procedures F1, F2, condition x, boolean done = false:

```
F1: S1; done = true; x.signal();
F2: if (!done) x.wait(); S2;
```

Monitor implementation with semaphores (full version)


```

semaphore mutex = 1;    // monitor entry
semaphore next = 0;     // signalers suspended here
int next_count = 0;     // # processes suspended on next

```

Each procedure P:

```

wait(mutex);
body of P
if (next_count > 0) signal(next); // resume a suspended signaler
else                signal(mutex); // or open the door

```

For each condition x: semaphore x_sem = 0; int x_count = 0;

```

x.wait():                x.signal():
x_count++;              if (x_count > 0) {
if (next_count > 0)      next_count++;
    signal(next);        signal(x_sem);
else signal(mutex);      wait(next);
wait(x_sem);             next_count--;
x_count--;               }

```

(The signaler steps aside onto next so the awakened process can run - preserving "one active process in the monitor.")

Resuming Processes within a Monitor

- If several processes are queued on condition x and x.signal() executes, which resumes? FCFS often isn't adequate.
- Conditional-wait: x.wait(c) where c = an integer priority number; the process with the lowest number (highest priority) is scheduled next.

Single Resource Allocation (ResourceAllocator monitor)

Allocate one resource among competing processes, priority = maximum time the process plans to use it (shortest time first):

```

monitor ResourceAllocator {
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy) x.wait(time);
        busy = true;
    }
    void release() {
        busy = false;
        x.signal();
    }
    initialization code() { busy = false; }
}

```

Usage: R.acquire(t); ... access resource ... R.release();

- Still vulnerable to incorrect use (release before acquire, double acquire, omissions) - same style of misuse as semaphores.

12. Liveness

- Processes may wait indefinitely trying to acquire a synchronization tool → violates progress and bounded waiting.
- Liveness = properties a system must satisfy to ensure processes make progress. Indefinite waiting is a liveness failure.

Deadlock

Two or more processes waiting indefinitely for an event that only one of the waiting processes can cause.

P0 wait(S); wait(Q); ... signal(S); signal(Q);	P1 wait(Q); wait(S); ... signal(Q); signal(S);	← each holds one, wants the other
---	---	-----------------------------------

P0 holds S and waits for Q; P1 holds Q and waits for S; neither will ever signal → deadlocked.

Starvation

- Indefinite blocking - a process is never removed from the semaphore queue it's suspended in. (System is live overall, but this process never proceeds.)

Priority Inversion

- A lower-priority process holds a lock needed by a higher-priority process, and a medium-priority process preempts the low one - the medium process indirectly blocks the high one.
- Scenario: P1 (high) waits for semaphore S held by P3 (low); P2 (medium) preempts P3 → P2 delays P3, and therefore P1.
- Solved via the priority-inheritance protocol: the process holding the resource inherits the priority of the highest-priority waiter until it releases the resource.

Key Terms Checklist

Race condition, critical section, entry/exit/remainder sections, mutual exclusion, progress, bounded waiting, interrupt disabling, turn variable, Peterson's solution, instruction reordering, memory model (strong/weak), memory barrier, test_and_set, compare_and_swap, atomic variable, mutex lock, acquire/release, busy waiting, spinlock, semaphore, wait/signal (P/V), counting vs binary semaphore, block/wakeup, monitor, condition variable, x.wait()/x.signal(), conditional wait (priority), ResourceAllocator, liveness, deadlock, starvation, priority inversion, priority inheritance.

Chapter 7: Synchronization Examples (Classical Problems)

(Silberschatz, Galvin & Gagne, 10th Edition)

1. Classical Problems of Synchronization

Used to test newly proposed synchronization schemes:

1. Bounded-Buffer Problem
2. Readers–Writers Problem
3. Dining-Philosophers Problem

These are the "benchmark puzzles" - if your new lock/semaphore/monitor idea can solve all three cleanly, it's probably sound.

2. Bounded-Buffer Problem (Producer–Consumer with semaphores)

Setup: n buffers, each holds one item. Three semaphores:

- $\text{mutex} = 1$ - mutual exclusion on the buffer itself
- $\text{full} = 0$ - counts filled slots
- $\text{empty} = n$ - counts empty slots

Producer:

```
while (true) {
    /* produce an item in next_produced */
    wait(empty);      /* claim an empty slot (block if none) */
    wait(mutex);      /* lock the buffer */
    /* add next_produced to the buffer */
    signal(mutex);    /* unlock */
    signal(full);      /* one more full slot */
}
```

Consumer:

```
while (true) {
    wait(full);        /* claim a full slot (block if none) */
    wait(mutex);      /* lock the buffer */
    /* remove an item from buffer into next_consumed */
    signal(mutex);    /* unlock */
    signal(empty);    /* one more empty slot */
    /* consume the item */
}
```

Key points:

- empty/full do the counting/waiting; mutex protects the actual buffer manipulation.
- Order matters: $\text{wait}(\text{empty})$ before $\text{wait}(\text{mutex})$. Reversed, a producer could lock the buffer then sleep waiting for an empty slot - and the consumer could never get the lock to create one → deadlock.

3. Readers–Writers Problem

- A data set is shared among concurrent processes:
- Readers - only read; no updates
- Writers - read and write
- Requirement: multiple readers may read simultaneously, but only one writer may access the data at a time (and with no readers present).
- Several variations exist - all involve some form of priorities.

Shared data (first readers–writers problem)

- Semaphore $\text{rw_mutex} = 1$ - writer exclusion (held by the writer, or by the reader group)
- Semaphore $\text{mutex} = 1$ - protects read_count
- Integer $\text{read_count} = 0$ - how many readers are currently reading

Writer:

```
while (true) {
    wait(rw_mutex);
    /* writing is performed */
    signal(rw_mutex);
}
```

Reader:

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1) /* first reader in */
        wait(rw_mutex); /* locks out writers */
    signal(mutex);
}
```

```

/* reading is performed */

wait(mutex);
read_count--;
if (read_count == 0)      /* last reader out */
    signal(rw_mutex);    /* lets writers in */
signal(mutex);
}

```

- The first reader locks `rw_mutex` on behalf of all readers; the last reader out releases it. Readers in between just increment/decrement the count.

Variations

- First readers–writers problem (above): readers favored → a writer may never write (starvation) if readers keep arriving.
- Second readers–writers problem: once a writer is ready, no newly arrived reader may start reading → readers can starve instead.
- Both can starve someone → more variations exist.
- Some kernels provide reader–writer locks directly.

4. Dining-Philosophers Problem

- N philosophers sit at a round table, a bowl of rice in the middle; they alternate thinking and eating; they don't interact with neighbors.
- To eat, a philosopher needs 2 chopsticks - one on each side, shared with neighbors - picked up one at a time; both released when done.
- With 5 philosophers, shared data: bowl of rice (data set) + semaphore `chopstick[5]`, all initialized to 1.
- It models a broad OS reality: many processes competing for multiple shared resources.

Semaphore solution (flawed)

```

while (true) {
    wait(chopstick[i]);          /* left */
    wait(chopstick[(i + 1) % 5]); /* right */
    /* eat for a while */
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
    /* think for a while */
}

```

The problem: if all 5 get hungry simultaneously and each picks up their left chopstick, everyone waits forever for a right chopstick held by a neighbor → deadlock.

Monitor solution

Idea: a philosopher picks up chopsticks only if both are available, checked atomically inside a monitor.

```

monitor DiningPhilosophers {
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();          /* both not available – wait */
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);          /* maybe left neighbor can now eat */
        test((i + 1) % 5);          /* maybe right neighbor can now eat */
    }

    void test(int i) {
        if ((state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING)) {
            state[i] = EATING;
            self[i].signal();
        }
    }

    initialization_code() {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}

```

Usage by philosopher i:

```

DiningPhilosophers.pickup(i);
/** EAT */
DiningPhilosophers.putdown(i);

```

- test(i): i may eat only if neither neighbor is eating and i is hungry.
- On putdown, the philosopher checks whether each neighbor can now eat and signals them.
- Result: no deadlock - but starvation is still possible (an unlucky philosopher's neighbors could alternate eating forever).

5. Kernel Synchronization — Windows

- Uniprocessor: uses interrupt masks to protect access to global resources.
- Multiprocessor: uses spinlocks; a spinlocking thread will never be preempted.
- Also provides dispatcher objects for user-land: mutexes, semaphores, events, timers.
- Events - act much like condition variables
- Timers - notify thread(s) when time expires
- Dispatcher objects are in signaled state (available - acquiring thread proceeds) or non-signaled state (thread blocks).
- Mutex dispatcher object: signaled = free; a thread acquiring it flips it to non-signaled; others block until it's signaled again.

6. Java Synchronization

Java provides: monitors, reentrant locks, semaphores, condition variables.

Java Monitors

- Every Java object has a single lock associated with it.
- Declaring a method synchronized → the calling thread must own the object's lock; if another thread owns it, the caller waits.
- The lock is released when the owning thread exits the synchronized method.
- A thread that tries to acquire an unavailable lock enters the object's entry set (threads waiting for the lock).

wait() and notify()

- Each object also has a wait set. When a thread calls wait():
 1. It releases the lock
 2. Its state becomes blocked
 3. It's placed in the wait set
- When a thread calls notify():
 1. An arbitrary thread T is picked from the wait set
 2. T moves from the wait set to the entry set
 3. T's state becomes runnable - it then competes for the lock and re-checks its condition
- Typical use: bounded buffer with synchronized insert()/remove(), calling wait() when full/empty and notify() after each operation.

Java Reentrant Locks

- Similar to mutex locks:

```
Lock key = new ReentrantLock();
key.lock();
try { /* critical section */ }
finally { key.unlock(); }
```

- The finally clause guarantees the lock is released even if an exception occurs in the try block.

Java Semaphores

```
Semaphore sem = new Semaphore(1); // constructor: initial count
sem.acquire();
try { /* critical section */ }
finally { sem.release(); }
```

Java Condition Variables

- Associated with a ReentrantLock; created via newCondition():

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition();
```

- A thread waits with await(), signals with signal().
- doWork example: five threads numbered 0–4, shared turn variable; a thread may work only when turn == its number. Using an array Condition[] condVars, each thread waits on its own condition variable (condVars[threadNumber].await()); on completion it sets turn = (turn + 1) % 5 and signals exactly the next thread (condVars[turn].signal()).
- Advantage over plain notify(): you wake the specific thread whose turn it is, not an arbitrary one.

7. Alternative Approaches

Transactional Memory

- A memory transaction = a sequence of read-write operations to memory performed atomically.
- Instead of update() guarded by mutex locks, write atomic { S } - the system ensures the statements in S execute atomically, no explicit locks, no deadlock risk from lock ordering.

Functional Programming Languages

- A different paradigm: they do not maintain state; variables are immutable - once assigned, they cannot change.
- If nothing can change, there's nothing to race on → data races become structurally impossible.
- Growing interest in Erlang and Scala for their approach to data races.

Key Terms Checklist

Bounded-buffer problem, empty/full/mutex semaphores, semaphore ordering deadlock, readers–writers problem, read_count, rw_mutex, first vs second variation, reader–writer locks, writer/reader starvation, dining philosophers, circular wait/deadlock, monitor solution (HUNGRY/EATING/THINKING, test()), starvation, Windows dispatcher objects, signaled/non-signaled, interrupt masks, spinlocks, Java monitor, synchronized, entry set, wait set, wait()/notify(), ReentrantLock, try/finally unlock, Java Semaphore acquire/release, Condition await()/signal(), transactional memory, atomic{S}, functional languages, immutability, Erlang/Scala.